LLM Skills and Metacognition — Sanjeev Arora (Princeton)

Distilled notes from a talk on how LLMs acquire, compose, and reason about skills.

1. Motivation

- Question: what are high-level ways to communicate to an LLM what we want it to do, beyond gradient descent on next-word prediction loss (and variants like SFT/RL)?
- Framing proposal: think in terms of **skills** rather than facts. A skill is a procedure; skills can be chained/composed.
- No clean formal definition of a skill exists, but we recognize them in practice.

2. Next-Word Prediction Requires Procedures

- Even if the word "metaphor" didn't exist, any entity that predicts next words well enough would implicitly learn the procedure for creating/understanding metaphors.
- Most complicated reasoning is procedural. Example prompt: "Give me two sentences in a fiction about sushi that exhibit metaphor and ad hominem attack." GPT-4o handles this.
- Composition is achieved without formal rules.

3. Skill-Mix Evaluation

- Randomly sample skills from a long list; ask the model to compose them in an unusual topic.
- Finding: GPT-4o could compose up to **5 skills** at a time. Llama-7B struggled with 2.
- Probability calculations (vs. estimated training data size) suggest most successful generations were novel, not memorized.

4. LLM Metacognition

- Metacognition = reasoning about one's own thought processes. LLMs exhibit this when asked.
- Three elicitation methods:

Method 1 — Task-based skill labeling

- Give task examples to a frontier LLM; ask it to label each with required skills in a constrained format (e.g., "four words joined by underscores").
- Feed the full label list back; ask for semantic clustering → hierarchical skill taxonomy.
- Applied to Hendrycks math → skills like `circle_properties_area_calculation`.

Method 2 — Direct elicitation of novel skills

- Prompt: "Give me a broad skill that has no existing name but which many humans will recognize."
- GPT-4o produced `linguistic exorcism` (rewording text to be less offensive/more useful). Not on Google.
- Follow-up: "Subskills of X?" → `synonym_substitution_technique`, `tone_moderation_adjustment`.
- Gemini 2.5 Pro (multimodal) produced `intuitive object placement` → subskills: edge-aware placement, stability assessment, implicit item clustering.
- Interpretation: these concepts exist as continuous structure in parameter space (because they're needed for next-word prediction); the prompt forces the model to condense them into discrete labels.

Method 3 — Comprehensive catalog generation

- "You are a great chat agent; give us instruction-following skills in this format."
- Produces ~1,000 skills with hierarchy.

5. Speculation: Why Does Next-Word Prediction Induce Metacognition?

- Open question. Arora's guess: an LLM is a **theory builder** for text pieces.
- To predict the next token, it must track multiple foci (e.g., a Winograd-style "city council" sentence has budget, environment, and consensus foci simultaneously).
- Recent Anthropic mechanistic-interpretability work found circuits that classify context into such classes for short contexts.
- Aside: analogous to a theory of consciousness — the brain has a "self-image" circuit (lesion evidence) that can be repurposed.

6. Applications of Skill Catalogs

Synthetic data for instruction-following

- Use GPT-4o to list ~1,000 instruction-following skills.
- Sample random skill pairs; prompt for a plausible user query + answer exhibiting both.
- **Only 4,000 Q&A pairs** trained Llama-3 8B (base) past Opus and 405B on AlpacaEval — vs. Alpaca's 50K or UltraChat's 1M.
- Why it works: the questions have many moving parts, forcing retrieval and composition.

Synthetic math data

- Random pairs of math skills → cross-topic questions (e.g., algebra + probability).
- All models drop in performance; empirically, $Y = X^2$ fits (probability of failing each skill compounds).

Data selection in Llama-3 paper

- Skill-relevant in-context examples improve k-shot performance; also cited for data selection during instruction tuning.

7. Arora–Goyal Theory of Skills (Pretraining)

Assumptions:

1. There is a set S of underlying skills; complex skills are compositions of r basic skills.
2. Pretraining data is a stream of finite text pieces; each text piece's comprehension relies on k skills.
3. Latent distribution over skills is a **product distribution** (independence assumption).

Main result (via random graph theory, plugging in Chinchilla exponents):

- **Scaling the model by 10× lets it comprehend text pieces with ~2× as many skills.**

Caveats: flat (no hierarchy), skills are independent latent variables, the 2× factor depends on the scaling-law exponent.

8. Training on Skill-Rich Data

Setup:

- Start with named language skills from Wikipedia (metaphor, ad hominem, etc.) — all models understand these as single skills.
- Generate SFT data combining k skills from a few categories (e.g., literary, rhetorical); hold out other categories.
- Evaluate composition of k' skills from **held-out** categories.

Findings:

- Training on k=2 improves held-out k=2, 3. Training on k=3 improves held-out k=3, 4, 5.
- The model learns compositionality **better than the flat statistical theory predicts** — generalizes to held-out skills.
- Question: can composing 2 skills bootstrap composition of more? From pretraining alone, no; but SFT unlocks the composer circuit.

9. Context-Enhanced Learning

Setup (distinct from vanilla SFT):

- Standard SFT: compute loss/gradients over (question, answer).
- Context-enhanced: also place **helpful extra information** (a "textbook") in the context, but do **not** compute loss/gradients over it.
- Helpful context appears only at training time — not at test time. Analogy: studying open-book, exam closed-book.

Synthetic task used:

- Multi-layer translation (input alphabet → output alphabet over d simple 2-symbol→2-symbol steps, with circular shifts).
- Each output symbol depends on 2^d input symbols.
- Theorem (SQ framework): learning this from input/output pairs alone requires exponential examples; empirically, SFT fails to learn it.

Method that works:

1. **Warm-up phase**: train on many random phrasebooks + productions so the model learns to pattern-match against the context.
2. **Target phase**: train on the single target phrasebook with **20% dropout** on the contextual "textbook" tokens.
3. At test time: 100% dropout (no textbook) — the model still solves the task.

Implications:

- The model internalizes the skill into its weights in a way that doesn't show up via next-token-prediction–based training-data detectors.
- Full dropout at train time doesn't work; in-context-learning capability is required; the paper includes mechanistic analysis of where the skill lives.

10. Open Research Questions

- Learning theory for metacognition — what mathematical framework captures it?
- How does training on data exhibiting skill 1 improve skill 2?
- How does the ability to compose skills emerge? What are limits on emergence speed (especially with think-tokens / RL)?
- Scaling laws for composition and SFT.
- Leveraging metacognition in training:
    - RL with skill-annotated rewards (higher-dimensional feedback signal).
    - Metacognition-based generalization of Constitutional AI (core skills → fleshed-out safety catalog).
    - Auto-formalization of math (Goedel-Prover: Lean tactics as skills of varying granularity).

11. Takeaways

- Models show increasingly sophisticated metacognition.
- Single-skill competence is widespread; **skill composition** is the bottleneck, improves with scale, and can be accelerated by carefully chosen SFT data.
- Compositional ability is a **safety concern**: models can combine skills in ways never seen in training. Focus on "was this behavior in the training data?" misses the compositional risk.